

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

```

```
df = pd.read_csv("house_purchase_dataset.csv")
```

```
df
```

	Age	Income	BuyHouse
0	60	34592	No
1	21	117196	Yes
2	37	52098	Yes
3	34	38289	No
4	26	108696	Yes
..	...	...	...
195	36	37361	No
196	42	29016	No
197	35	68434	Yes
198	38	40676	No
199	48	91200	Yes

```
[200 rows x 3 columns]
```

```
df.head()
```

	Age	Income	BuyHouse
0	60	34592	No
1	21	117196	Yes
2	37	52098	Yes
3	34	38289	No
4	26	108696	Yes

```
df.tail()
```

	Age	Income	BuyHouse
195	36	37361	No
196	42	29016	No
197	35	68434	Yes
198	38	40676	No
199	48	91200	Yes

```
df.shape
```

```
(200, 3)
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Age         200 non-null    int64
1   Income      200 non-null    int64
2   BuyHouse    200 non-null    str
dtypes: int64(2), str(1)
memory usage: 4.8 KB
```

```
df.describe()
```

	Age	Income
count	200.000000	200.000000
mean	41.375000	69647.205000
std	13.170485	29491.222968
min	20.000000	20074.000000
25%	30.000000	44756.500000
50%	40.000000	69703.500000
75%	53.000000	95888.500000
max	65.000000	119943.000000

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df["BuyHouse"] = df["BuyHouse"].map({
    "Yes":1,
    "No":0
})
```

```
df.head()
```

	Age	Income	BuyHouse
0	60	34592	0
1	21	117196	1
2	37	52098	1
3	34	38289	0
4	26	108696	1

```
df.corr()
```

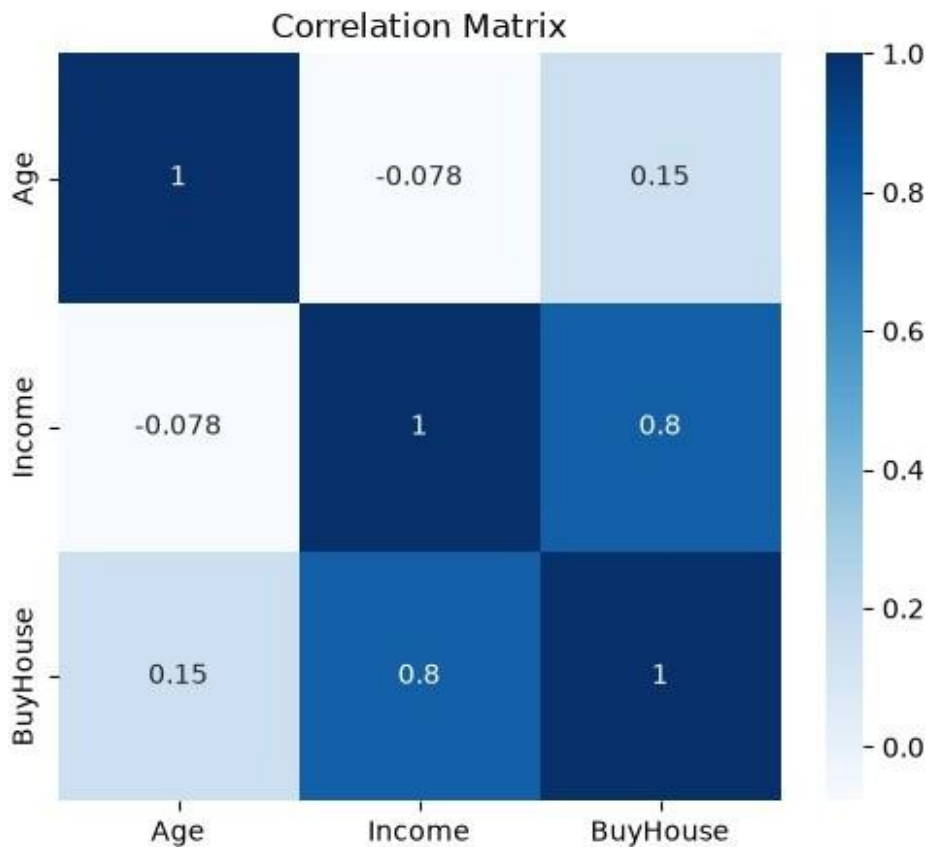
	Age	Income	BuyHouse
Age	1.000000	-0.078370	0.150719
Income	-0.078370	1.000000	0.799536
BuyHouse	0.150719	0.799536	1.000000

```
plt.figure(figsize=(6,5))

sns.heatmap(df.corr(),
            annot=True,
            cmap="Blues")

plt.title("Correlation Matrix")

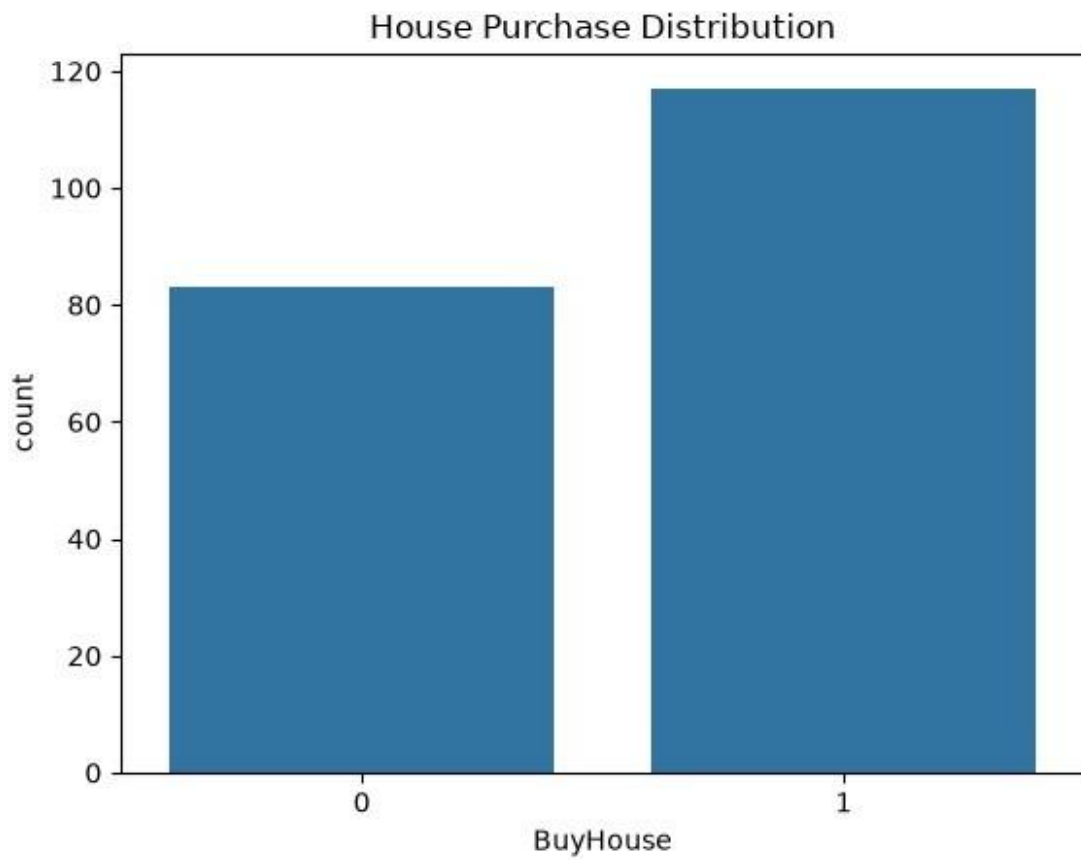
plt.show()
```



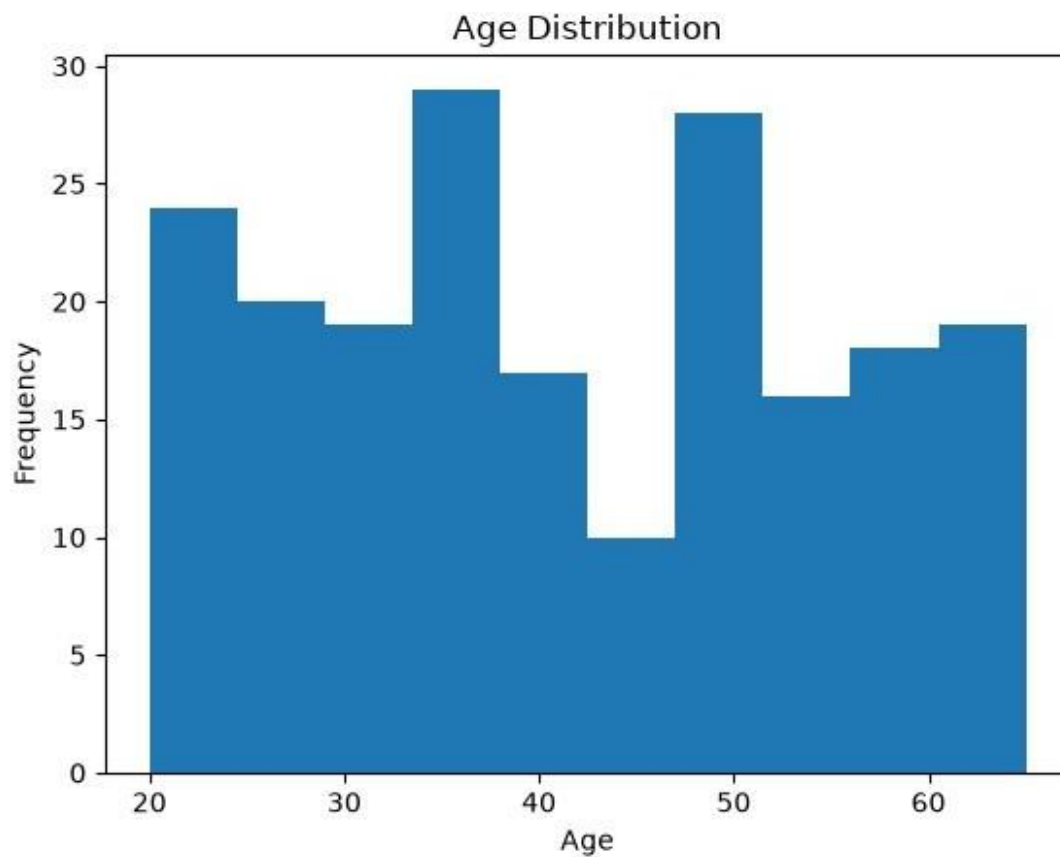
```
sns.countplot(x="BuyHouse",data=df)

plt.title("House Purchase Distribution")

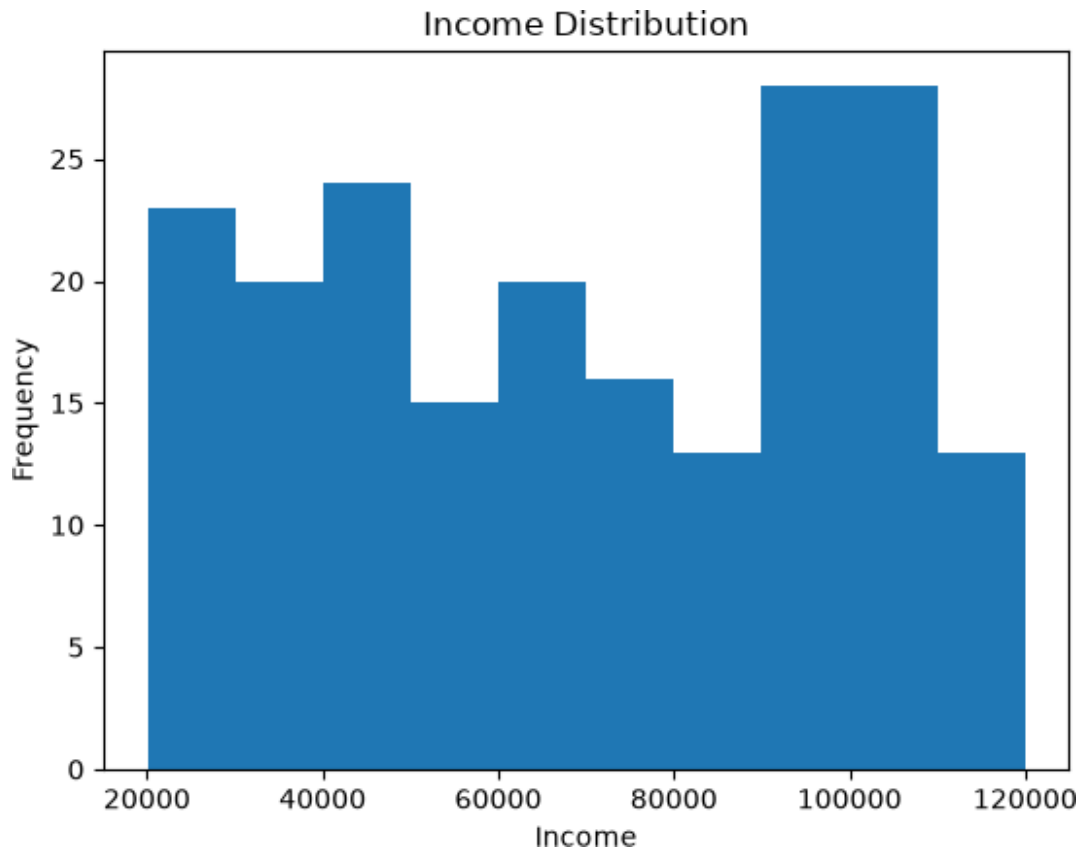
plt.show()
```



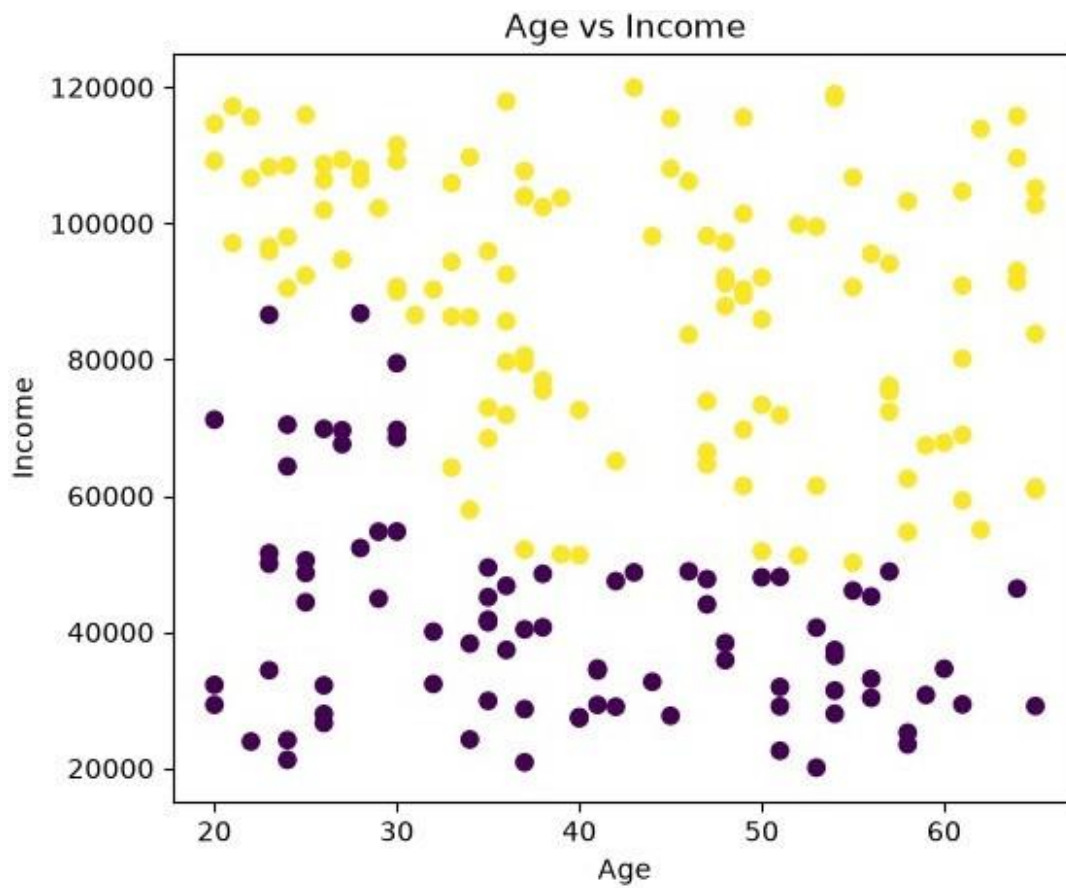
```
plt.hist(df["Age"],bins=10)
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.title("Age Distribution")
plt.show()
```



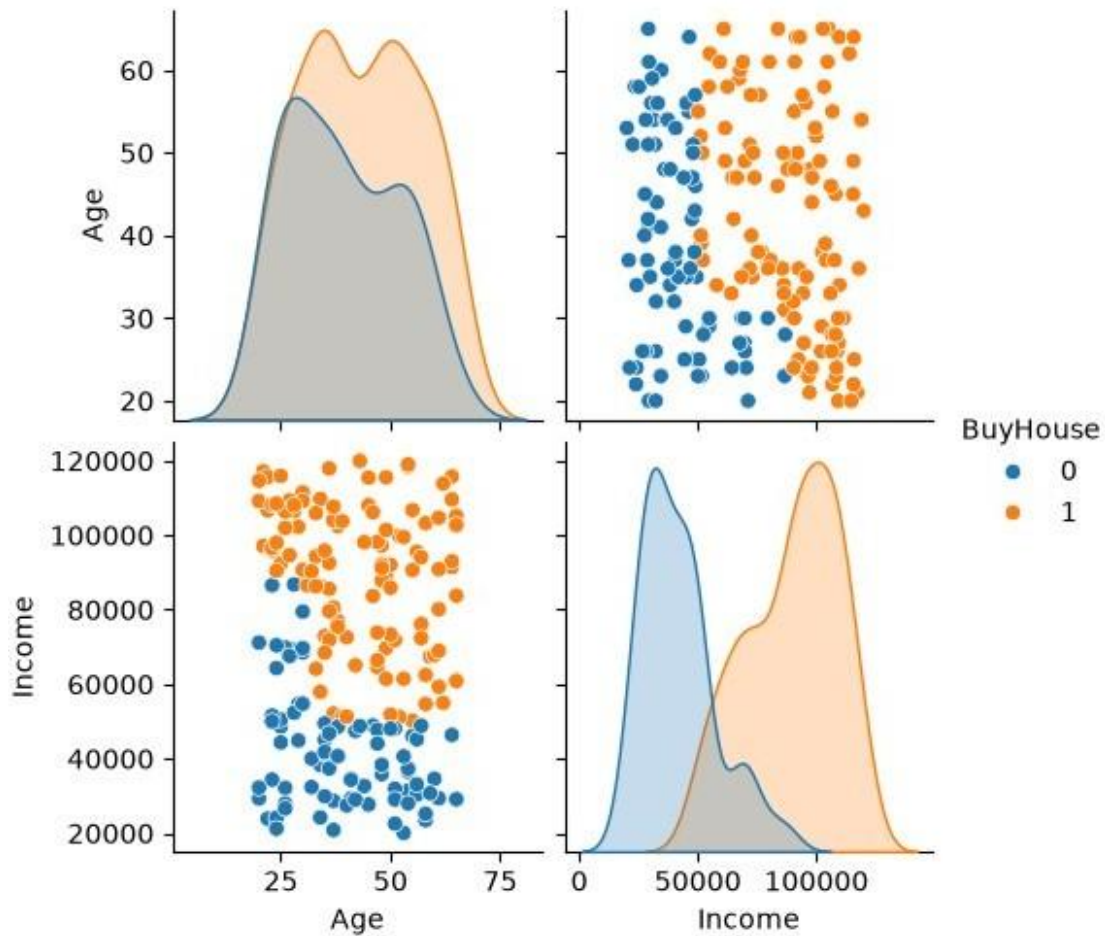
```
plt.hist(df["Income"],bins=10)
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.title("Income Distribution")
plt.show()
```



```
plt.figure(figsize=(6,5))  
plt.scatter(df["Age"],  
            df["Income"],  
            c=df["BuyHouse"])  
  
plt.xlabel("Age")  
plt.ylabel("Income")  
plt.title("Age vs Income")  
plt.show()
```



```
sns.pairplot(df, hue="BuyHouse")  
plt.show()
```



```
X = df[["Age", "Income"]]
```

```
y = df["BuyHouse"]
```

```
X.head()
```

	Age	Income
0	60	34592
1	21	117196
2	37	52098
3	34	38289
4	26	108696

```
X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
y.head()
```

```

0      0
1      1
2      1
3      0
4      1
Name: BuyHouse, dtype: int64

print(X_train.shape)

print(X_test.shape)

print(y_train.shape)

print(y_test.shape)

(160, 2)
(40, 2)
(160,)
(40,)

model = DecisionTreeClassifier(random_state=42)

model.fit(X_train,y_train)

DecisionTreeClassifier(random_state=42)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test,y_pred)

print("Accuracy =",accuracy)

Accuracy = 1.0

cm = confusion_matrix(y_test,y_pred)

print(cm)

[[15  0]
 [ 0 25]]

plt.figure(figsize=(5,4))

sns.heatmap(cm,
             annot=True,
             fmt="d",
             cmap="Greens")

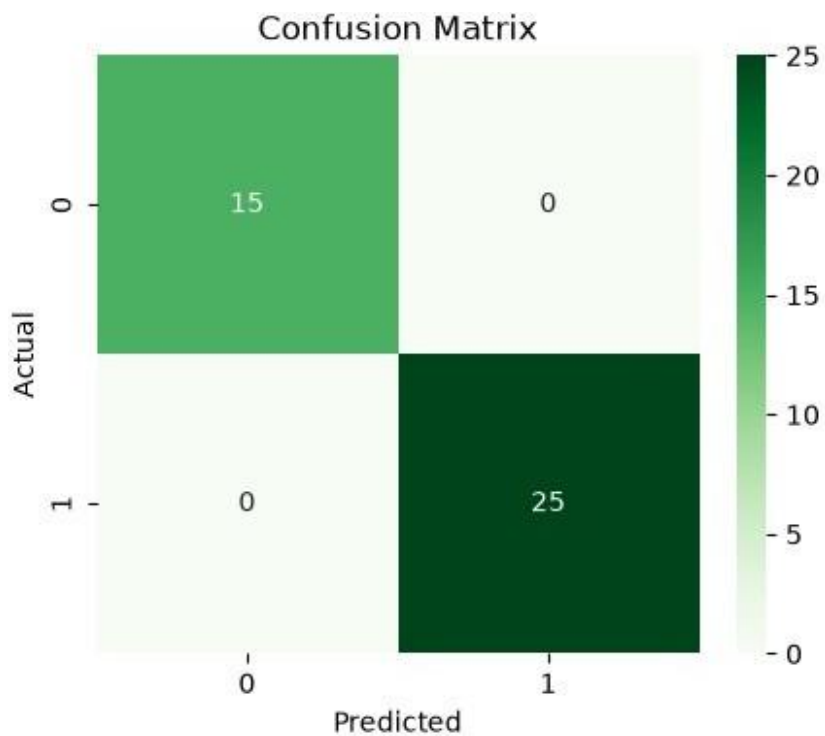
plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

```

```
plt.show()
```



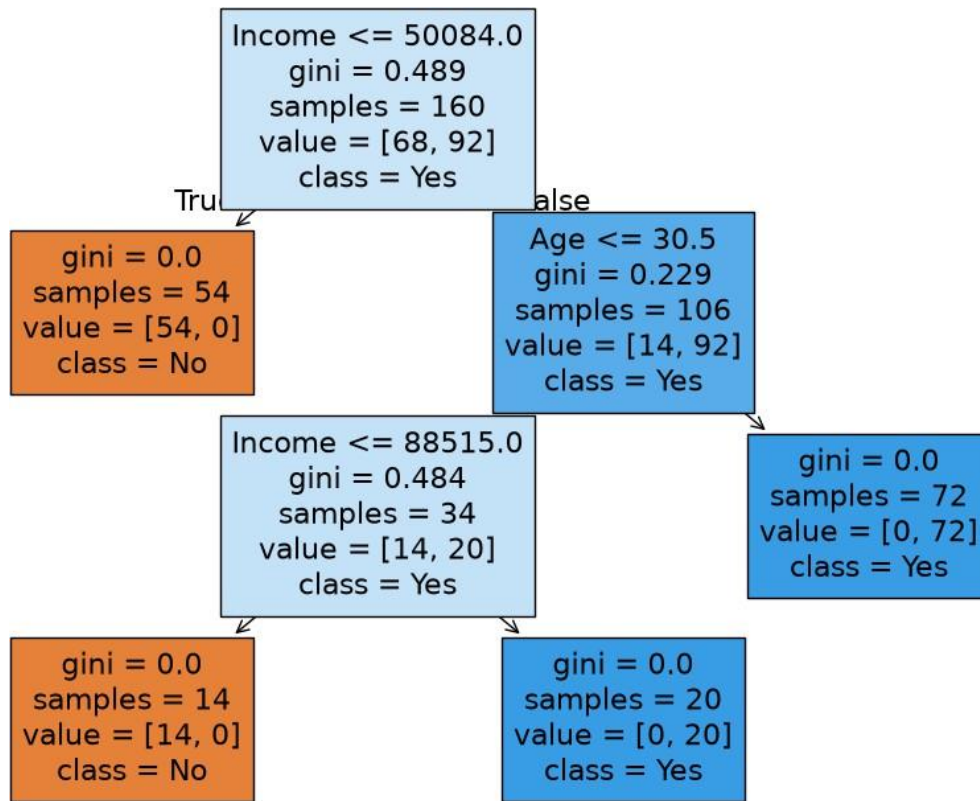
```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	15
1	1.00	1.00	1.00	25
accuracy			1.00	40
macro avg	1.00	1.00	1.00	40
weighted avg	1.00	1.00	1.00	40

```
plt.figure(figsize=(12,8))
```

```
plot_tree(model,
          feature_names=X.columns,
          class_names=["No", "Yes"],
          filled=True)
```

```
plt.show()
```



```

importance = pd.DataFrame({
    "Feature":X.columns,
    "Importance":model.feature_importances_
})

print(importance)

  Feature  Importance
0     Age    0.100144
1  Income    0.899856

person = pd.DataFrame({
    "Age":[35],
    "Income":[70000]
})

prediction = model.predict(person)

print("Prediction:",prediction)

Prediction: [1]

if prediction[0]==1:
    print("Person is likely to buy a house.")

```

```
else:
    print("Person is not likely to buy a house.")

Person is likely to buy a house.

print(model.predict_proba(person))

[[0. 1.]]

results = pd.DataFrame({
    "Actual":y_test,
    "Predicted":y_pred
})

results.head()

   Actual  Predicted
95       0          0
15       1          1
30       1          1
158      1          1
128      0          0

results.to_csv("House_Purchase_Predictions.csv",index=False)

print("Prediction file saved successfully.")

Prediction file saved successfully.
```